SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I R.Jagan-192321102 *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: R.Jagan

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model

iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

ii. Explain how backtracking search works in this scenario

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Answers:

1.AI Drones for Emergency Medicine Delivery

Problem summary: A drone must reach a flood-affected location by the fastest and safest route. Roads become blocked suddenly, terrain/weather change movement cost, and only a heuristic (straight-line distance) plus partial real-time updates are available. Evaluation functions: Greedy f(n) = h(n); A* f(n) = g(n) + h(n), where g(n) is the actual cost so far and h(n) the estimated cost to goal.

(a) Behaviour of Greedy Best-First Search (GBFS)

GBFS always expands the node that appears closest to the goal (smallest straight-line distance h(n)) and ignores the cost already spent. The drone will therefore head almost directly towards the hospital/relief camp.

- Strengths: Very fast and memory-light; makes quick decisions, which is useful when urgency is high; finds a path quickly when the terrain is open and the heuristic is reasonably good.
- Weaknesses under uncertainty: Not optimal and not complete in graphs with loops (without a closed list). It ignores terrain/weather cost, so it may choose a "short" path through a flooded valley or storm zone. It can be lured into dead-ends (e.g., a straight-line-close area whose road is blocked) and must then back-track; it has no notion of risk.

Example: If a river lies directly between the drone and the camp, GBFS keeps moving toward the river bank (smallest h) even though crossing is impossible, and only later discovers the detour.

(b) How A* handles the situation

A* expands the node with the lowest f(n) = g(n) + h(n). Here g(n) captures the real, variable cost (distance × terrain factor × weather penalty, plus a risk penalty for storm areas), while h(n) steers the search toward the goal.

- Balancing: g(n) prevents the search from choosing cheap-looking but expensive routes; h(n) prevents the search from wandering in all directions as uniform-cost search would.

A heavily weather-affected edge raises g and makes A* automatically prefer a slightly longer but calmer route.

- Properties: If h(n) is admissible (never over-estimates) and consistent, A* is complete and optimal. Straight-line distance is admissible only if scaled by the minimum possible cost per unit distance, i.e. h(n) = d_straight × c_min.
- Cost: Time and memory can grow exponentially in the worst case, but a good heuristic prunes most of the search space.

(c) Impact of heuristic accuracy

Heuristic quality	Greedy Best-First	A*
Accurate (close to true cost)	Very fast, near-direct route; quality still not guaranteed.	Expands very few nodes; returns optimal path.
Under-estimates (admissible but weak, e.g. h=0)	Behaves almost randomly/blindly.	Stays optimal but degenerates towards Dijkstra – slow.
Over-estimates (inadmissible)	Little change – already ignores g; may go wrong direction faster.	May miss the optimal path; result becomes sub-optimal (acts like weighted A*), but is faster.
Misleading (ignores floods/blocks)	Severely affected – commits to bad region.	Partly protected by g(n); recovers after finding real costs.

Thus GBFS is highly sensitive to heuristic accuracy, whereas A* is more robust because the g(n) term corrects heuristic errors.

(d) Effect of dynamic changes (blocked paths)

- A path computed earlier may become invalid when a bridge or road is blocked; the stored f-values and the open/closed lists become stale.
- GBFS recovers quickly by re-selecting the next best-looking node, but without any cost awareness – it may repeatedly walk into new blocked regions.

- A* must re-plan: re-running it from the current position is correct but costs time. Incremental variants – D* Lite, LPA*, Anytime Repairing A* – reuse earlier computations and only update affected nodes, which suits frequent changes.
- More frequent changes lower the usefulness of any long pre-computed plan; the drone should plan over a short horizon, re-plan each time sensor data changes, and treat blocked edges as infinite cost.

(e) Recommendation and justification

Recommended: A* (preferably its dynamic/anytime variant, D* Lite or Weighted A*) with an admissible, terrain-scaled heuristic.

Criterion	Justification
Real-time constraint	Weighted/Anytime A* ($f = g + w \cdot h$, $w \approx 1.2–1.5$) gives a good route fast and improves it if time permits; D* Lite re-plans incrementally.
Optimality	With an admissible heuristic A* returns the least-cost route, minimizing delay; GBFS gives no guarantee.
Safety	Risk (storm zones, no-fly areas, battery reserve) can be added into g(n), so dangerous routes are avoided; GBFS ignores this.
Uncertainty	Re-planning when blocked paths are reported keeps the route valid; g(n) corrects heuristic errors.

GBFS can be used only as an emergency fallback when computation time is almost zero. Overall, A*-family algorithms give the best balance of speed, optimality and safety for life-critical medicine delivery.

2.Smart City Traffic Signal Optimization (Local Search & Optimization)

Problem summary: Hundreds of intersections must be co-ordinated to minimize waiting time, fuel use and congestion. The search space is huge, traffic is dynamic and algorithms may get stuck in local optima.

(a) Formulation as an optimization problem; why traditional search is inefficient

Component	Formulation
State	A complete signal plan S = {(green time g_i, cycle length C_i, offset o_i, phase order) for every intersection i = 1…N}.
Initial state	Current/pre-set timing plan (or random plan).
Actions / neighbours	Change the green time of one phase by ±Δ, shift an offset, swap phase order at one or a few intersections.
Objective function	Minimize $J(S) = w_1 \cdot$(Total waiting time) + $w_2 \cdot$(Fuel consumption) + $w_3 \cdot$(Congestion/queue length); weights reflect city policy. Equivalent to maximizing −J.
Constraints	Minimum/maximum green time, pedestrian crossing time, cycle length limits, emergency-vehicle priority, no conflicting greens.
Evaluation	Traffic simulator or real sensor data gives J(S) for any plan.

Why traditional search (BFS, DFS, A*) is inefficient: (i) the state space is astronomically large – if each of N = 300 intersections has only 10 possible timings, there are 10^{300} plans; (ii) there is no single goal state, only a "best-possible" value, so path cost is irrelevant; (iii) evaluating one state needs simulation, which is expensive; (iv) storing the search tree is impossible in memory; (v) the environment changes, making any stored plan outdated. Hence local search / metaheuristics, which keep only the current state(s) and improve them, are appropriate.

(b) Hill Climbing in this scenario and its limitations

Hill Climbing starts from the current plan, evaluates neighbouring plans (e.g., adding 2 seconds of green at one junction) and moves to the neighbour with the best improvement in J (steepest-ascent), stopping when no neighbour is better.

```
current = initial_plan
```

```
loop:
  neighbour = best_of(neighbours(current))   # lowest J
  if J(neighbour) >= J(current): return current
  current = neighbour
```

- Advantages: simple, low memory, quick improvements, easily run online.
- Limitations: greedy and short-sighted; gets stuck at local optima, plateaus and ridges; result depends strongly on the starting plan; no way to move to a worse state to find a better one; not complete and not optimal.

(c) Local maxima, plateaus and ridges – real-world interpretation

Issue	Meaning	Traffic example
Local maximum (minimum of J)	A state better than all neighbours but worse than the global best.	A "green wave" along the main avenue reduces delay there, but any single-junction change increases delay; a better city-wide plan that sacrifices the avenue slightly is never reached.
Plateau	A flat region where neighbouring states have equal J.	During off-peak hours, changing green time by 2 s changes waiting time negligibly, so the algorithm cannot choose a direction and wanders or stops.
Ridge	A narrow high-value path that cannot be climbed by single-step moves.	Improvement needs coordinated changes (green length AND offset at adjacent junctions); changing only one parameter at one junction makes things worse, so the ridge is missed.
Dynamic shift	The landscape itself changes.	An accident or a match crowd alters flow, so yesterday's optimum becomes a poor state.

(d) How Simulated Annealing and Genetic Algorithms overcome these limitations

Simulated Annealing (SA): picks a random neighbour; if it is better it is always accepted; if worse it is accepted with probability P = e^(−ΔJ / T). The temperature T starts high (much exploration) and is gradually reduced (cooling schedule) so the search becomes greedy at the end. Accepting occasional worse plans lets the system escape local optima, cross plateaus and traverse ridges.

Genetic Algorithm (GA): maintains a population of signal plans (chromosome = list of timings). Steps: evaluate fitness = 1/J → selection of fitter plans → crossover (combine timing blocks of two plans, e.g., different city zones) → mutation (random timing tweak) → new generation. Because many regions of the space are explored in parallel and good building blocks are recombined, GA is much less likely to get trapped and naturally handles coordinated changes (ridges).

Aspect	Simulated Annealing	Genetic Algorithm
Search unit	One solution	Population of solutions
Escape mechanism	Probabilistic acceptance of worse moves	Crossover + mutation + selection diversity
Memory/computation	Low	Higher (many evaluations per generation)
Parallelism	Limited	Excellent (evaluate individuals independently)
Adaptation to change	Re-heat temperature	Keep population, continue evolving

(e) Recommended approach and justification

Recommendation: a hybrid scheme – Genetic Algorithm for global exploration, followed by Simulated Annealing/Hill Climbing for local fine-tuning (memetic approach), run continuously in a rolling-horizon manner.

- Scalability: GA evaluates individuals in parallel on a city cluster/cloud; the city can be divided into zones (each a sub-chromosome), so hundreds of intersections remain tractable.

- Adaptability: the existing population is kept as a "warm start"; when traffic changes, evolution continues from good plans instead of starting over. SA is re-heated mildly for quick local adjustments during rush hour.
- Solution quality: global search avoids poor local optima; local fine-tuning gives accurate final timings.
- Practicality: the multi-objective J can be weighted (or handled with a Pareto-based GA such as NSGA-II); a safe fallback plan (fixed timing) is always available.

Plain Hill Climbing is only suitable as the final polishing step, not as the main algorithm.

3.Autonomous Mars Rover (Online Search Agents)

Problem summary: The rover explores unknown terrain with limited sensors, must avoid craters and rocks and has no complete map. It decides while acting and learns as it moves.

(a) Why an online search agent instead of offline search

An offline search agent computes a complete plan before acting; this needs a full, known model of the environment (map, costs, results of actions). On Mars the map is unknown, so no complete plan can be made. An online search agent interleaves computation and action: it acts, observes the new percept, updates its knowledge and then decides the next action.

- The environment is unknown/exploration problem – the rover learns the outcome of actions only by performing them.
- Sensors see only a limited radius, so only local information is available at each step.
- Communication delay with Earth (4–24 minutes one way) prevents remote step-by-step control, so the rover must decide autonomously and promptly.
- Offline planning with a wrong assumed map could lead the rover into craters – irreversible failure.

(b) Challenges of partial observability and dynamic environments

Challenge	Explanation
Partial observability	Camera/lidar range is limited, shadows hide holes, sensors are noisy; the rover cannot know what lies beyond its view, so the current state is not fully known (belief state needed).
Uncertainty of actions	Wheel slip in sand may cause the rover to move less than intended – the result of an action is not deterministic.
Dynamic environment	Dust storms, sliding sand, changing light and temperature; obstacles may appear or terrain may collapse.
Irreversible actions	Falling into a crater or toppling over cannot be undone – dead-end/unsafe states must be avoided.
Resource limits	Limited energy, memory, computing power and time; exploration must be efficient.

Challenge	Explanation
Localization error	Position estimate drifts without GPS – map and position must be estimated together (SLAM).

(c) Building and updating the internal model

The rover maintains a model that grows as it explores:

- Map / occupancy grid: each cell is labelled free, obstacle, hazard or unknown (often with a probability). Newly sensed cells update the map.
- Result table RESULT(s, a) = s′: records what actually happened when action a was taken in state s – this builds the transition model.
- Cost/heuristic table H(s): current estimate of the cost to the goal for each visited state (used by LRTA*); updated after each move using H(s) = min over actions [c(s,a,s′) + H(s′)].
- Visited / untried-action lists: for systematic exploration and back-tracking (as in Online DFS).
- Position estimation: sensor readings and odometry are fused (e.g. SLAM, Kalman/particle filters) to keep the position and map consistent.

Cycle: Sense → update model → select action using current model → act → repeat. If a predicted route proves blocked, the cost of that edge is updated and the plan is revised.

(d) Comparison of online search with uninformed and informed search

Feature	Uninformed (BFS/DFS/UCS)	Informed (A*, Greedy)	Online search (Online-DFS, LRTA*)
Knowledge needed	Full state space	Full state space + heuristic	Only local percepts; learns the model
Planning	Offline	Offline	Interleaved with action
Heuristic use	None	Fixed heuristic guides search	Heuristic learned/updated at run time (LRTA*)

Feature	Uninformed (BFS/DFS/UCS)	Informed (A*, Greedy)	Online search (Online-DFS, LRTA*)
Handles unknown terrain	No	No	Yes
Cost	Computation is expensive (exponential)	Less expansion but needs map	Real movement cost, small computation per step
Safety in real world	Plans on assumed map	Plans on assumed map	Adapts as hazards are discovered; back-tracking possible if actions are reversible
Optimality	BFS/UCS optimal on known graph	A* optimal with admissible h	Not optimal initially; improves with repeated trials (competitive ratio measure)

(e) Most suitable approach and justification

Recommended: an online informed search – LRTA* (Learning Real-Time A*) / D* Lite on an incrementally built map – combined with a safety layer.

- Uncertainty: operates with partial knowledge, assuming free space for unknown cells (optimism) and correcting when sensors show otherwise.
- Adaptability: the heuristic table/map is updated after every step, so route quality improves and new hazards are handled by local re-planning (D* Lite repairs only affected portions).
- Safety: hazard cells get very high/infinite cost, and safety margins around craters are enforced; unsafe actions are rejected by a reactive safety module; the rover only commits to short moves within the sensed, verified region. Energy-aware cost functions and a "safe-return" fallback are included.
- Efficiency: small per-step computation suits the rover's limited processor.

Pure offline A* or uninformed search cannot work because the map does not exist beforehand; a purely reactive method would be myopic. Hence online informed search with safety nstraints is the most appropriate.

4.University Exam Timetable Generation

Problem summary: Assign each exam to a time slot and hall so that all hard constraints (no student clash, hall capacity, prerequisites order, faculty availability, special accommodations) are satisfied; complexity increases with the number of courses and students.

(a) Formulation as a CSP

A CSP is defined by a triple (X, D, C) – variables X, domains D and constraints C. A solution is a complete and consistent assignment that satisfies every constraint.

- X: one variable per exam (course) – E_1, E_2, …, E_n (optionally also invigilator variables).
- D: each exam can take values from D = {(day, time-slot, hall)} – for example, 10 days × 2 slots × 8 halls = 160 values per exam (reduced by unary constraints).
- C: the restrictions listed in part (b).

The problem can be pictured as a constraint graph in which nodes are exams and an edge joins two exams that share students, a hall or a faculty member.

(b) Variables, domains and constraints

Constraint (type)	Formal expression	Explanation
No student overlap (binary, hard)	If $Students(E_i) \cap Students(E_j) \neq \emptyset$ then $time(E_i) \neq time(E_j)$	A student cannot write two exams at the same time.
Hall capacity (global/binary, hard)	For exams in the same slot: Σ students $\leq$ capacity(hall); two exams in same slot cannot share a hall unless capacity allows	Limited exam halls; seating limits.
Precedence (binary, hard)	$time(E_i) < time(E_j)$ when E_i must precede E_j	E.g., Core subject before advanced subject.
Faculty availability (unary, hard)	$time(E_i) \in$ AvailableSlots(faculty)	Reduces the domain directly (node consistency).
Special accommodation (unary/hard)	$hall(E_i) \in$ AccessibleHalls; extra-time exams need longer slots	Wheelchair access, extra time, separate room for some students.

Constraint (type)	Formal expression	Explanation
Soft constraints (preference)	Minimize back-to-back exams, avoid >2 exams/day	Used for optimization after feasibility; violations are penalized.

Variables: exams; domains: (day, slot, hall) triples after pruning by unary constraints; hard constraints must hold, soft constraints improve quality.

(c) Backtracking search in this scenario

Backtracking search is a depth-first search that assigns one variable at a time, checks consistency with the assignments made, and undoes (backtracks) the last assignment when no value is possible.

```
BACKTRACK(assignment):
 if complete(assignment): return assignment
 var = SELECT-UNASSIGNED-VARIABLE()       # e.g. MRV
 for value in ORDER-DOMAIN-VALUES(var):    # e.g. LCV
   if consistent(var=value, assignment):
      add var=value; result = BACKTRACK(assignment)
      if result != failure: return result
      remove var=value
 return failure
```

Illustration: Assign Maths → Day 1, 9 AM, Hall A. Next, Physics shares students with Maths, so it cannot use Day 1, 9 AM; assign Day 1, 2 PM, Hall A. If later "Advanced AI" (must be after AI Basics and needing an accessible hall) has no legal slot, the algorithm backtracks to the most recent choice and tries another value. Used plainly it may waste time on partial schedules already doomed to fail, so heuristics are added:

- MRV (Minimum Remaining Values): choose the exam with the fewest legal slots (most constrained first).
- Degree heuristic: choose the exam with most conflicts (largest enrolment/shared students) when ties occur.
- LCV (Least Constraining Value): try the slot that rules out the fewest options for the neighbours.

(d) Constraint propagation (forward checking) and efficiency

Forward checking: whenever an exam is assigned a (day, slot, hall), all unassigned neighbouring exams have inconsistent values removed from their domains immediately. If any domain becomes empty, the search backtracks at once.

Example: After Maths is fixed at Day 1, 9 AM, forward checking removes that slot from every exam sharing students with Maths; if "Physics" is left with an empty domain the failure is detected before further assignments.

- Benefits: early failure detection, smaller domains, far fewer nodes explored; combines well with MRV (domain sizes are always up-to-date).
- Stronger propagation: Arc Consistency (AC-3) or maintaining arc consistency (MAC) also propagates the effect of removed values between unassigned variables (e.g., precedence chains), detecting conflicts that forward checking misses – at higher cost per node.
- Other speed-ups: node/unary consistency before search, conflict-directed backjumping, and symmetry-breaking (identical halls).

(e) Real-world challenges and best strategy for scalability

Challenge	Handling
Large size (thousands of exams/students)	Decompose the problem (by faculty/department, via constraint-graph structure); cluster exams into slots first using graph colouring.
Over-constrained – no feasible solution	Convert some constraints to soft ones; use weighted CSP / minimize violations; request manual relaxation.
Late changes (withdrawal, hall unavailable)	Use local repair (min-conflicts) starting from the existing timetable rather than re-solving from scratch.
Fairness and student welfare	Add soft constraints/penalty functions (gaps between exams, number per day).
Computation time	Time-limited search, randomized restarts, parallel solving.

Best strategy: a hybrid approach – model as CSP; apply preprocessing (node and arc consistency); solve with backtracking + MRV/Degree/LCV + forward checking/MAC to obtain a feasible timetable, and then improve it using local search (min-conflicts, simulated annealing or tabu search) for soft constraints and for fast repairs when changes occur. This combines the completeness of systematic search with the scalability of local search.

5.Strategic Game AI – Minimax and Alpha-Beta Pruning

Problem summary: The AI opponent in a real-time strategy game competes with human players, must decide within strict time limits and faces an extremely large game tree.

(a) How Minimax models the problem

Minimax models the game as a two-player, adversarial, (assumed) zero-sum game tree:

- State: the complete game situation (positions of units, resources, buildings, map control).
- Players: MAX = the AI (tries to maximize utility), MIN = the human (assumed to play the best counter-moves to minimize the AI's utility).
- Actions: legal moves (move/attack/build/research) producing successor states.
- Terminal test and utility: win/loss/draw, or a heuristic evaluation at the depth cut-off.

Value propagation: MINIMAX(s) = Utility(s) if s is terminal; max of the children's values at MAX nodes; min of the children's values at MIN nodes. The AI selects the move leading to the child with the highest backed-up value, thus guaranteeing the best result against a perfect opponent. (For real-time play, the continuous game is discretized into decision steps/turns, and imperfect information is handled by assumptions or sampling.)

(b) Computational challenges of large game trees

- Exponential growth: time complexity $O(b^d)$, space $O(b \cdot d)$ (depth-first), where b = branching factor and d = search depth. In an RTS with $b \approx 30–100$ legal actions, even depth 6 means $10^9–10^{12}$ nodes.
- Strict time limits: decisions needed in milliseconds; full search to terminal states is impossible.
- Expensive evaluation: each state has many units and interactions; evaluating and generating states costs time and memory.
- Real-time and simultaneous actions: players act simultaneously and continuously, which makes the pure turn-based Minimax assumption only an approximation.
- Imperfect information (fog of war) and non-optimal human opponents reduce the accuracy of the model.
- Horizon effect: depth-limited search may miss a disaster just beyond the cut-off.

(c) How Alpha-Beta pruning improves efficiency

Alpha-Beta pruning returns exactly the same move as Minimax but skips branches that cannot influence the final decision. It keeps two bounds during depth-first search:

- α (alpha): best (highest) value MAX can already guarantee along the current path.
- β (beta): best (lowest) value MIN can already guarantee along the current path.

Pruning rule: if $\alpha \geq \beta$ at any node, the remaining children of that node are cut off.

Detailed reasoning (example): Suppose MAX at the root has explored the first move and obtained value 6, so $\alpha = 6$. In the second move (a MIN node), the first child gives 3, so $\beta = 3 \leq \alpha$. Since MIN can force a value ≤ 3 there, and MAX already has 6 from the first move, MAX will never choose the second move – the other children of that node need not be evaluated, whatever their values. This saves the entire sub-tree below them.

Case	Complexity / effect
Worst case (poor move ordering)	$O(b^d)$ – no pruning, same as Minimax.
Best case (perfect ordering)	O(b^(d/2)) – search is effectively twice as deep in the same time (or the square root of the nodes).
Average case (random ordering)	About O(b^(3d/4)).

Improving pruning: move ordering (try captures/high-value attacks first, killer-move and history heuristics), transposition tables (cache already evaluated states reached by different move orders), and iterative deepening (shallow search results give a good ordering for deeper search).

(d) Design of the evaluation function and justification

When depth/time limit is reached, the evaluation function Eval(s) estimates how good the state is for the AI. A weighted linear function is commonly used:

$Eval(s) = w_1 \cdot Military + w_2 \cdot Economy + w_3 \cdot Map\ Control + w_4 \cdot Technology + w_5 \cdot Safety/Threat + w_6 \cdot Mobility$

Factor	Measured by	Justification
Military strength (w_1)	Own army value (units × strength) − enemy army value; health of units	Wins battles; the strongest predictor of winning.
Economy (w_2)	Resources collected, number of workers/production buildings	Long-term power to build units and tech.
Map/territory control (w_3)	Controlled resource points, vision area, strategic locations	Controls resource flow and limits enemy options.
Technology (w_4)	Upgrades researched, advanced units unlocked	Provides future advantage.
Safety/threat (w_5)	Base health, enemy units close to base, vulnerability of key buildings	Avoids losing the game suddenly; handles horizon effect.
Mobility/options (w_6)	Number of legal/advantageous moves available	Flexibility to respond and to attack.

Design notes: the function must be fast to compute (called millions of times), consistent with true utility at terminal states (win = $+\infty$, loss = $-\infty$), and weights should be tuned by self-play, machine learning (e.g., regression on game records) or genetic algorithms. The weights can be made phase-dependent (economy emphasized early, military later). Non-linear terms can capture interactions (e.g., army × tech).

(e) Strategy for balancing optimality and real-time performance

A perfect (optimal) search is infeasible, so the AI should aim for the best possible decision within the time budget:

- Depth-limited Alpha-Beta + Iterative Deepening: search depth 1, 2, 3… until the time limit; always keep the best move of the last completed depth (anytime behaviour).
- Good move ordering + transposition table to maximize pruning; killer/history heuristics.
- Selective search/forward pruning: discard obviously bad moves (beam search, null-move/futility pruning); extend critical lines (quiescence search, e.g., ongoing battles) to reduce the horizon effect.

- Abstraction: group units into squads and use high-level macro-actions (attack, expand, defend) to lower the branching factor; use scripted behaviours for low-level actions.
- Parallel search and time management: allocate more time in critical situations, less in obvious ones.
- Hybrid methods: Monte-Carlo Tree Search (MCTS) or learned policy/value networks can replace or complement Minimax in very large/uncertain games.

Conclusion: Iterative-deepening Alpha-Beta with a well-designed evaluation function, strong move ordering and selective pruning gives near-optimal decisions while meeting real-time limits; the depth or abstraction level can be adjusted dynamically according to the time available.